Google Cloud H2S

AI Builder Cup

Build in JAPAC.
Build for the world.



## Team Details

Team name: TL

Team leader name: Taivanbat.B · Member: Lkhagvadorj.M

Problem Statement: BFSI — Intelligent Risk, Fraud & Financial Experiences. Kervan Credit Passport: an AI underwriting agent that turns a small business's informal records (ledger photos, bank statements, payment exports) into a verified, explainable credit passport a bank can act on. Demo market: Mongolia.

# Brief about the idea

## The problem

- Most Mongolian micro and small businesses keep their books in a handwritten ledger, a bank account and a QR payment app. Banks cannot underwrite that at scale, so thin-file businesses are refused or overpriced.
- Declared figures are unverifiable, manual review is slow, and decisions are opaque to the owner.

## Kervan Credit Passport

- The owner uploads a ledger photo, a bank statement (PDF/CSV) and a payment export, and declares revenue, tenure, amount and tenor.
- In about a minute the agent returns a credit passport (grade K1–K7 with an indicative PD, rate band and suggested limit from Kervan's v0.1 policy scorecard, to be recalibrated on partner-bank data), an owner explanation in English and Mongolian with a re-scored improvement plan, and an underwriter-ready credit memo with an evidence appendix.
- Core principle: Gemini reads, verifies, explains and writes; points, penalties and grade come only from a deterministic, auditable scorecard. Gemini never computes a score and can never raise one: at most it tightens an input's verification status, and its flags carry no penalty.
- Banks receive anonymized passports and make indicative offers; identity is revealed only when the business accepts, and the bank's own KYC and credit approval follow before anything is disbursed.

# Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

## How it differs

• Verification, not extraction: declared vs documented and documents vs each other (ledger vs bank, payments vs bank, totals vs re-summed rows, account holder, tenure vs account age); arithmetic in Python; every figure traced to a page or row.

• Explainable by construction: a points scorecard with reason codes; self-declared inputs count half, inconsistent inputs count zero.

• Reads the records people actually have: handwritten Mongolian ledger photos, local bank CSV/PDF exports, payment settlements.

## How it solves the problem · USP

• Banks: an underwriter-ready memo and evidence trail in a minute, inconsistencies flagged; every memo is a draft for a human underwriter (K1–K2 need sign-off, K3–K6 are referred, K7 gets no standard offer).

• Owners: a plain-language grade in Mongolian and an improvement plan with gains computed by re-running the scorer.

• USP: trust by design — number guardrails, no protected attributes, cross-source consistency checks today (authenticity and registration documents on the roadmap), identity disclosure controlled by the business.

# List of features offered by the solution

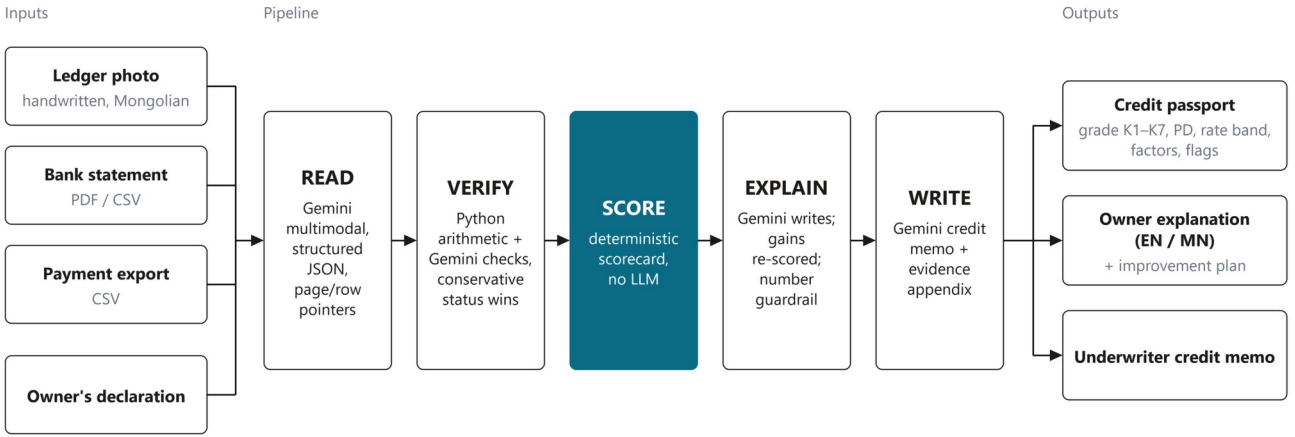
## Agent pipeline (live, streamed to the UI)

- READ: Gemini multimodal, structured JSON; monthly figures with document + page/row pointers.
- VERIFY: Python re-sums CSVs and checks continuity, volatility, declared-vs-bank, ledger-vs-bank, payments-vs-bank, account age; Gemini adds qualitative checks; the more conservative status wins.
- SCORE: deterministic scorecard, no LLM; per-factor points, reason codes, penalties.
- EXPLAIN: owner summary in English and Mongolian; improvement plan with re-scored gains.
- WRITE: credit memo with cash-flow analysis, risks, mitigants, scorecard table and evidence appendix.

## Product views

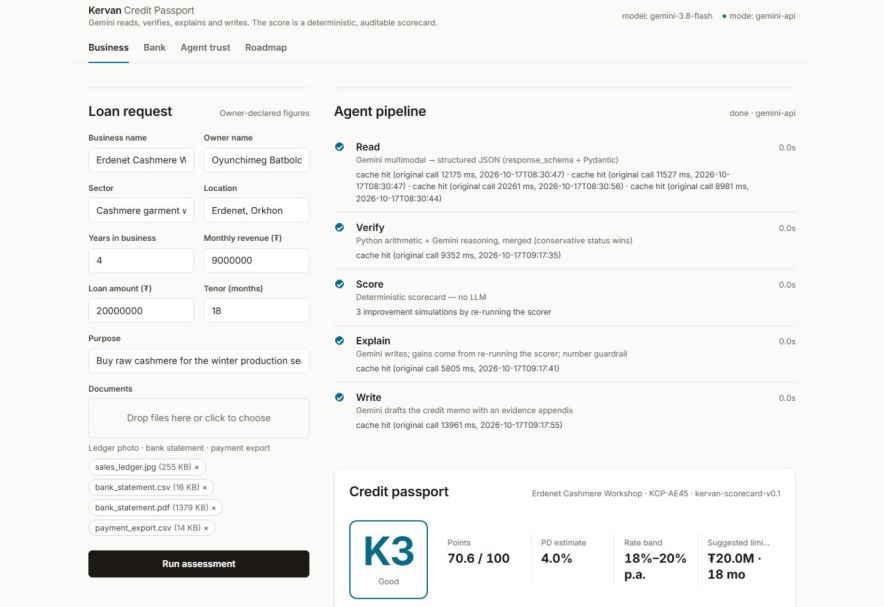
- Business: loan request, uploads, live pipeline, passport card with status-encoded factor bars, flags, EN/MN toggle, improvement plan, offers.
- Bank: anonymized passport list, flags, memo, make-offer; identity revealed only when the business accepts.
- Agent trust: guardrails, step trace (prompt version, model, latency, schema validation, cache), scorer inputs, verification checks, evidence trail with links to the documents.

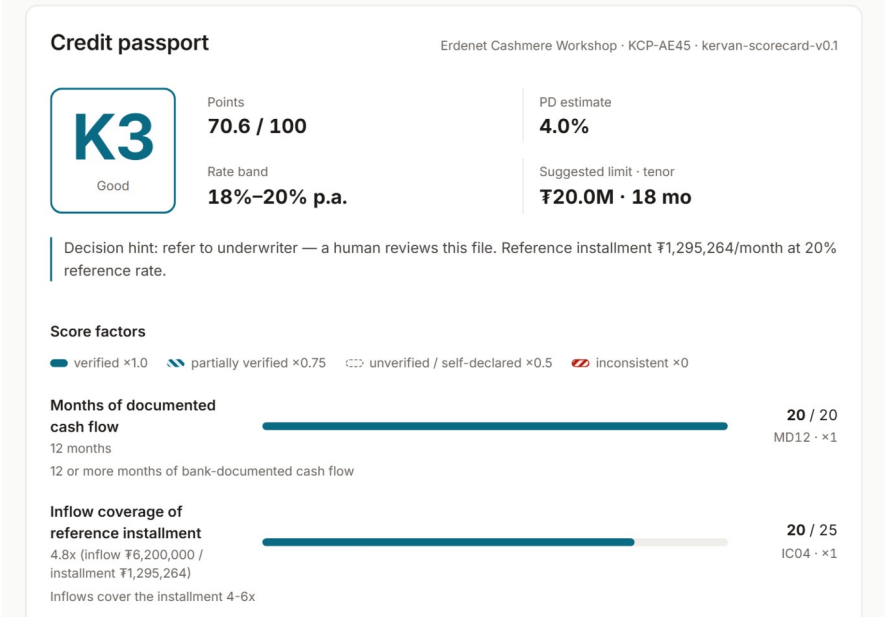
# Process flow diagram or Use-case diagram




Gemini reads, verifies, explains and writes. The score comes only from the scorecard.

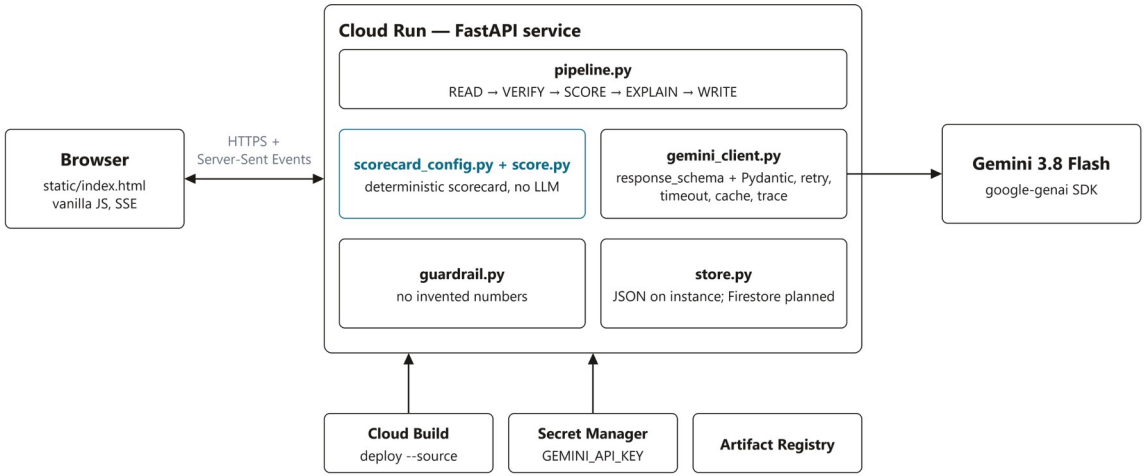
# Wireframes/Mock diagrams of the proposed solution (optional)




Left: loan request and live agent pipeline. Right: the credit passport with status-encoded factor bars, flags and the re-scored improvement plan.

# Architecture diagram of the proposed solution




Browser (static HTML/JS served by FastAPI) → Cloud Run service → Gemini 3.8 Flash via google-genai (structured JSON, one call per document in parallel). Build: Cloud Build from source → Artifact Registry. Secrets: Secret Manager → env at runtime. Progress streams back over Server-Sent Events.

# Technologies to be used in the solution

## Google Cloud and AI

- Gemini 3.8 Flash via the google-genai SDK: JSON output with a Pydantic response_schema on all four model steps; low thinking level for sub-minute runs; one retry feeding the validation error back; per-call timeout; cache keyed by model, prompt version and file hash. The same code runs on Vertex AI with the Cloud Run service account (no API key).
- Cloud Run: deploy from source (Cloud Build → Artifact Registry), 1 vCPU / 1 GiB, scales to zero. Secret Manager holds the Gemini key; the runtime service account has only secretAccessor; no secret in the repo or image.
- Storage: JSON in /tmp on the Cloud Run instance for the demo (resets on cold start); Firestore persistence for passports, offers and traces is the planned next step.
- Access: the demo service is public and holds synthetic data only; Cloud Run IAM or Identity-Aware Proxy gates it before real customer documents.

## Application stack

- Python 3.12, FastAPI, Pydantic v2; Server-Sent Events for live progress.
- Deterministic verification math and a points scorecard in one config file; unit tests pin the behaviour.
- Number guardrail: every figure in generated text must exist in the inputs; retry once, then deterministic fallback.
- Front-end: one static page, vanilla HTML/CSS/JS.
- Prompts versioned as text files with the guardrails inside.

# Estimated implementation cost (optional)

## Per assessment (prototype measurements)

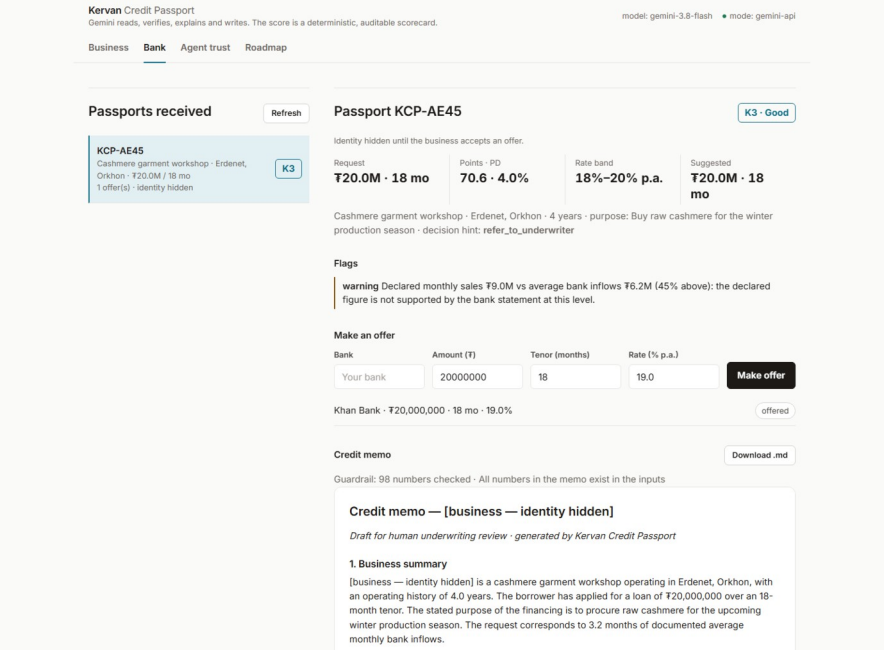
- 4 to 7 Gemini Flash calls per assessment (one per document plus verify, explain, write); 60–120k input tokens; about US$0.02–0.05 at Flash list prices. Repeated runs on the same files are served from the cache at zero cost.
- Cloud Run with scale-to-zero: compute only while a request runs (~1 vCPU-minute per assessment); within the free tier for pilot volumes.

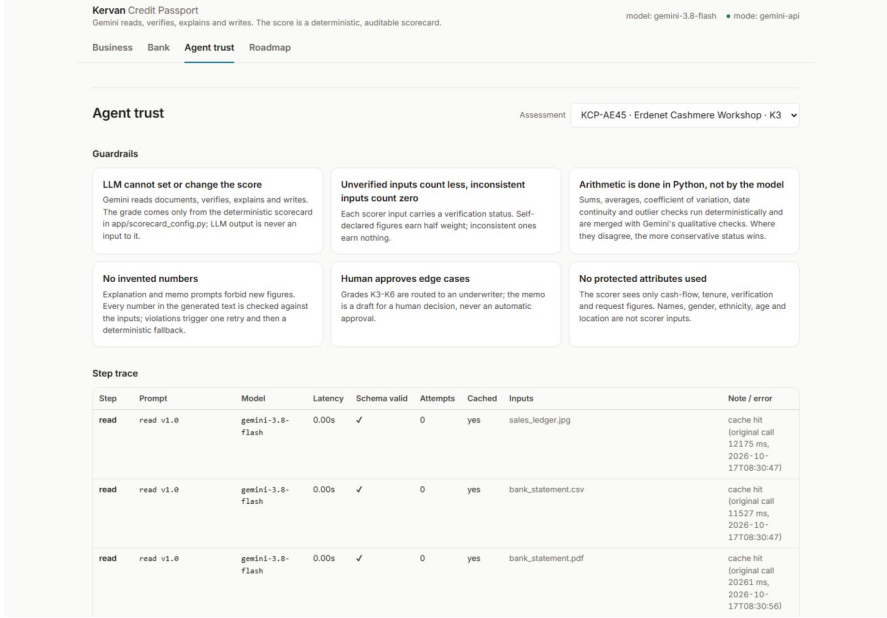
## Monthly, indicative

- Pilot (1,000 assessments / month): ≈ US$30–60 Gemini + ≈ US$5–15 Cloud Run + Secret Manager/Artifact Registry cents.
- Scale (50,000 assessments / month): ≈ US$1.5–2.5k Gemini + ≈ US$150–300 Cloud Run; Firestore adds cents per thousand operations.
- No per-seat licences; the scorecard and verification run in plain Python.

Estimates use public list prices in October 2026 and the prototype's measured token volumes; actual cost depends on document sizes.

# Snapshots of the prototype




Left: Bank view — anonymized passport, offer, Gemini-drafted credit memo. Right: Agent trust — guardrails, step trace and evidence trail.

# Prototype Performance report/Benchmarking

| Measure | Result (live Gemini 3.8 Flash, prototype) |
|---|---|
| **End-to-end run, 4 documents, cold** | ≈ 60 s wall time (reads in parallel); 25–40 s for 1–3 documents |
| **Per step latency** | READ 4–20 s per document · VERIFY ≈ 9 s · SCORE < 5 ms · EXPLAIN ≈ 6 s · WRITE ≈ 14 s; repeated runs < 25 s (document reads cached) |
| **Extraction accuracy on synthetic fixtures** | 12/12 months exact per document type (Pillow-rendered ledger photo, bank CSV, bank PDF, payment export) on three invented businesses; hackathon rules: no real data; not yet measured on real customer documents |
| **Schema validation** | 100% of model calls validated on the first attempt in test runs |
| **Number guardrail** | ≈ 100 numbers checked per memo, 0 invented figures in test runs |
| **Verification** | 18 checks per assessment; declared-vs-bank gap flagged (−3 points) · thin file flagged |
| **Tests** | Unit tests for the scorer, verification math, uploads and storage pass |
| **Scorecard calibration** | Not yet calibrated: Kervan v0.1 weights, PD estimates and rate bands are expert-set initial policy, pinned by tests, to be recalibrated on partner-bank portfolio data |

# Additional Details/Future Development (if any)

## Next 90 days

- Firebase Authentication for business and bank logins (ID tokens verified in FastAPI) and Identity-Aware Proxy in front of the bank view; Cloud Storage with signed URLs for uploaded documents.
- Registration and tax documents to fully verify tenure and identity; e-signature consent for data sharing.
- Scorecard calibration on portfolio data with a partner bank; monitoring of grade distributions and overrides; step traces to Cloud Logging.
- Gemini through Vertex AI so data stays in-region; Cloud Build trigger on main running the tests before each deploy; bank-side API into loan-origination systems.

## Scale story

- Same pipeline for Kazakhstan, Uzbekistan and Kyrgyzstan: a per-market scorecard config and prompt language notes; schemas, verification math, guardrails and deployment stay the same.
- Owners carry their passport between lenders and refresh it as records grow — the credit file follows the business, not the bank.

Provide links to your:

- GitHub Public Repository
- Demo Video Link (3 Minutes)
- Final Product Link

**GitHub public repository**

https://github.com/taivnaaiwnl/singsing

**Demo video (3–4 minutes)**

(demo video link)

**Final product link (Cloud Run)**

(Cloud Run URL)

# Kervan Credit Passport

Gemini reads, verifies, explains and writes. The score comes only from the scorecard.

Team TL · Taivanbat.B · Lkhagvadorj.M

Google Cloud H2S

# AI Builder Cup

Build in JAPAC.
Build for the world.

# Thank you

